# Teaching the sorting factory workshop

22 October 2026 · Four hours · First semester · No prerequisites

Students learn by predicting a small change, running it and explaining a visible result. Use the student notebook as the main route. The instructor notebook adds guidance beside identical executable activities. Swap typing and predicting roles every activity. Red routes left, blue right, all other reports reject.

## Prepare before class

Prepare Python 3.11 or newer in an environment separate from ROS. Install notebooks/requirements.txt in that environment before class. Extract the complete student ZIP; retain exercises, notebooks and workshop together. Do not distribute the instructor notebook as learner material. Open a clean notebook, run setup and the first activity, save a team copy and demonstrate recovery after a kernel restart.

```
python3 -m venv .venv-workshop
```

```
.venv-workshop/bin/python -m pip install -r notebooks/requirements.txt
```

```
.venv-workshop/bin/python notebooks/launch.py
```

Windows uses .venv-workshop\Scripts\python.exe with the same launcher. The launcher opens a local JupyterLab with the bundle as its root. No JupyterLite service is provided. Internet is needed for the external Blockly Maze and Teachable Machine websites; check the links and prepare offline alternatives in workshop-resources.md. The bundled Blockly factory runs offline after setup; follow docs/blockly-student.md for the learner challenge and docs/blockly-workshop.md for instructor setup. Use nonpersonal objects, not student faces, for camera examples. Print one worksheet per pair.

## Timetable and transitions

| Elapsed minutes | Task | Teaching move |
|---|---|---|
| **00–25** | Introduction | Ask how humans sort; show sensing, deciding, acting and feedback; demonstrate notebook controls |
| **25–35** | Blockly Maze | Levels 1–3 core; 4–5 stretch; reserve last minute to return to notebook |
| **35–65** | A Python | Greeting 6 min, repeat 8, condition 10, explanation and transition 6 |
| **65–85** | B Teachable Machine | Classes 5 min, tests 7, changed condition 5, explanation 3 |
| **85–95** | Break | Save; name three groups and starting stations |
| **95–140** | C Stations | Three 15-minute rotations including 1-minute transitions |
| **140–190** | D Factory | Trace 10 min, baseline 12, change rule 15, compare routes 10, save 3 |
| **190–225** | E Blockly factory or notebook challenges | Choose 5 min, run and record 22, exchange tests 8 |
| **225–240** | Demos | Prepare 4 min, show results 8, reflect and save 3 |

Exactly 240 minutes. Do not add the live lab as an extra lesson. The camera notebook contains optional extensions: select one controlled observation experiment within the 20-minute perception block; leave further list experiments for challenges. Definitions follow visible examples, not a vocabulary lecture.

## Beginner prompts and solutions

A: show Shift+Enter, output, quotes and indentation. A variable stores a value with =; == compares values. Original loop prints 1, 2, 3. Five repeats print 1–5. Red chooses left; blue right;

yellow reject. If a quote is missing, read the last error line, repair one quote and rerun. A kernel restart clears values but preserves saved edits; run setup again.

B: prepared red, blue and background classes reduce setup time. Keep a few examples out of training. Ask what changes when background or illumination changes. The notebook photograph crops are real photographs with human descriptions; its report lists are controlled teaching data, not output of an implemented camera classifier. Actual red reported blue goes right; None rejects. A plausible incorrect report cannot be detected by the routing rule alone. Offline use the photograph and actual/report comparison; avoid presenting demo counts as recognition accuracy.

D: baseline is left 2, right 2, reject 1, with five belt advances and actions left/right/left/reject/right. Swapping rules has identical totals for this balanced input but different actions. Use three reds and one blue to reveal the difference in counts. Stop precedes observation and placement. Adding green without a rule rejects it; with a green-to-left rule it reaches left. None, unknown and invalid destinations reject. Counting every object does not prove every route is correct.

E: pairs choose the offline Blockly factory challenge or notebook challenges for the same 35-minute slot. For Blockly, start in simulation: predict and run, change the sequence, add color branches, test unknown, then save and explain the result. Use the instructor Blockly guide for setup and recovery. For notebooks, select two experiments, reset the original inputs between them and record predictions before running. Adding None, unknown, 42 and empty text to the baseline yields left 2, right 2, reject 5. Red-to-middle produces left 0, right 2, reject 3. Ten reports must produce ten outcomes. Ask early finishers for a test that detects swapped rules.

## Three stations

Use three groups: robot → camera → virtual; camera → virtual → robot; virtual → robot → camera. Each slot uses explain 2 min, try 9, explain evidence 3, save and rotate 1.

Robot and twin: open Robot_Candy_Factory_Live_SO101.ipynb. It defaults to invented DEMO data. Students choose one body joint, read a graph and calculate one relative versus absolute prediction. Relative mapping is follower start + leader now − leader start; 30 + 15 − 10 = 35 degrees. Absolute prediction is 15 degrees. Angle differences can be expected in relative mode. The gripper JointState is a visual proxy and is excluded. A twin follows measurements; it is not an independent confirmation of reality.

A supervised physical Blockly extension replaces time within the robot rotation or challenge slot; it adds no lesson time. After simulation, teach a short fixed-jaw stroke with the leader, record the follower, return to the checked starting pose and replay once at speed 0.5. Add one approved EV3 action only when commissioned; require unknown to stop before manipulation. Dispensing and jaw control need their own commissioning. For payload sorting, the separate Run one commissioned sorting cycle block lets the native station own the complete grasp and object ownership transaction; a taught replay plus jaw movement does not establish a grasp. Review setup and recovery in docs/blockly-workshop.md.

Camera and conveyor: trace move → stop → observe → decide → pick using paper objects or a qualified instructor demonstration. Ask why stopping before picking matters. Change one visual condition and explain uncertainty. No student connects motor hardware or changes station controls.

Virtual robot: run the notebook's short destination sequence; compare order and totals. Original left/right/left has the same totals as left/left/right. Add reject, then middle; invalid destinations reject. Optional kinematics activity is linked in workshop-resources.md; use the sequence fallback if unavailable. A destination label is not a physical motor coordinate.

## Live observer preparation

Use the full checkout on the same station host as ROS. Keep the existing commissioned resident controller; the observer starts no controller, opens no USB, calls no ROS services and reads only /leader/joint_states, /follower/joint_states and /follower/teleop_status. Real enable, mode selection and stop belong in the instructor's RViz operations panel. Review current physical acceptance evidence before demonstrating motion.

```
# Offline invented examples, no ROS needed
.venv-workshop/bin/python tools/launch_live_workshop.py
# Instructor only, existing station topics through existing ROS wrapper
ROS_DOMAIN_ID=76 .venv-workshop/bin/python tools/launch_live_workshop.py --live
```

The helper defaults to this checkout's existing ros2/conda/so101-conda.sh run wrapper. It uses the prepared Conda ROS environment, while Jupyter stays isolated. For the remote station pass --ros-wrapper ~/.local/bin/station-fwmini4 --ros-wrapper-command ros-vulkan to the live launcher. This preserves the station's accepted environment. No container image is needed. The live Jupyter server uses loopback port 8890, separate from the student server on 8888; a second launch reports that the port is in use. Jupyter remains a normal Python kernel. Each capture starts a bounded subscriber through the existing prepared ROS wrapper; it collects for 3 seconds (maximum 5), samples at most 5 times/second and exits, destroying the node. The outer process timeout includes up to 20 seconds for ROS wrapper startup. This avoids installing incompatible ROS libraries into Jupyter. Localhost binding and normal Jupyter token authentication are preserved. For a remote browser use the prepared SSH forwarding arrangement; do not expose an unauthenticated notebook server.

Freshness requires receipt age and source age at most one second. Joint source age is ROS header age; the follower header is publication time, not an independent motor acquisition timestamp. The controller retains its own acquisition freshness gates. Teleop status uses same-host monotonic time; latched disabled status may be old. The notebook hides expired values, reports missing streams and never silently substitutes demo data. A fresh topic does not establish motion permission, independent geometry accuracy or physical sorting success.

If a LIVE stream disappears, rerun the observation after the instructor restores the existing station. Every call starts a new node; reconnection needs no notebook service. On failure use the explicit DEMO launcher and restart the kernel. Never infer zero from missing data. The observer is additional visualization, not an emergency stop or safety monitor.

## Offline and environment fallback

Run python3 exercises/check_setup.py, then exercises/01_python.py, 03_vision.py, 02_robot.py and 04_sort.py in lesson order. Edit the marked STUDENT AREA, save and rerun. Python standard library is sufficient. Run exercises/05_visual_sort.py and open sorting-result.html for an offline browser animation. If computers fail, use paper objects, report cards and three bins with the same predictions and worksheet.

## Build and verification

Build notebooks and shared handbook text with python tools/build_notebooks.py. Build printable handouts using SOFFICE=/path/to/prepared/soffice python tools/build_materials.py --handouts-only. Build the learner ZIP with python exercises/make_student_bundle.py --output docs/student-workshop.zip. Execute tools/verify_notebooks.py in the prepared environment; it opens fresh kernels and checks meaningful variants. These software checks do not establish physical commissioning or complete sorting acceptance.